

The Singularity Workshop - FSM API (Advanced)

This package contains an advanced and highly flexible integration of the FSM API with Unity. It is designed for developers who need fine-grained control over their state-driven logic, particularly in complex projects.

Accessing Documentation & Samples

The FSM API is best understood through its demos, which showcase core concepts like FSM reusability and concurrent state processing.

To access the complete list of demos and the full User Guide:

1. **Open the Unity Package Manager.**
2. **Find the FSM API-Advanced package.**
3. **Click the "Samples" button** to import the desired documentation and demo scenes into your project.

Available Demo Samples:

Sample Name	Core Concept Demonstrated
Simple Light Demo - Advanced	Basic FSM setup and controlling simple object state.
Simple Rotation Demo	Reusable FSM logic controlling the rotation of multiple independent objects.
Simple Scalar Demo	Reusable FSM logic for scaling along configurable single or multi-dimensional axes.
Simple Translation Demo	Reusable FSM logic for constrained linear motion.
Rotation/Scalar/Translation	FSM Composition: Running multiple, non-conflicting FSM instances concurrently on a single GameObject.

Pure Integration & Upgrade Guide

This guide provides instructions for integrating the FSM API into an existing Unity project.

NOTE: This package requires **TheSingularityWorkshop.FSM_API v1.0.11 or greater.**

Upgrading from the Basic Package

Upgrading from the basic package is designed to be seamless.

Delete the basic FSM_UnityIntegration prefab from your scene.

Find the new advanced integration prefab and drag it into your scene.

Drag the root of your FSM domain into the integration component.

By default, the advanced prefab is configured to be identical to the basic integration, ensuring your existing code remains functional.

Using Process Groups

The advanced package introduces Process Groups, which allow you to organize and update specific state machines with precision. Instead of a single update loop for all state machines, you can define which groups of FSMs get updated in specific Unity messages.

For example, you can create a `PlayerProcessGroup` to handle all player input logic in Unity's `Update` method, a `UpdateLogicProcessGroup` for core AI behaviors in `FixedUpdate`, and a `StatusTrackingProcessGroup` to manage production queues or other non-critical state tracking.

To configure your Process Groups:

On the advanced integration prefab, locate the lists for Unity messages su

Add the string names of your desired Process Groups to these lists.

The integration will automatically manage and run the specified groups dur

♥ Support Our Vision on Patreon

